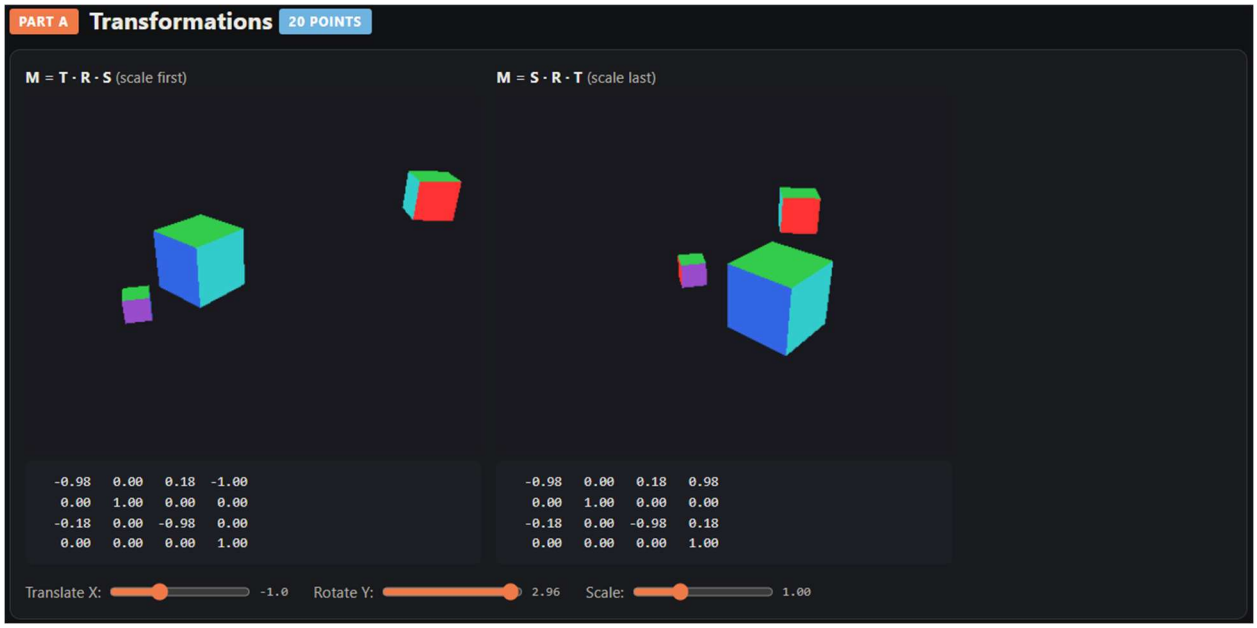
Painting Selection & Overview

- **Chosen Painting:** Şeker Ahmed Paşa — *Kavunlu Natürmort* (Sakıp Sabancı Museum Collection).
- **Rationale:** *Kavunlu Natürmort* features prominent, smooth organic contours—specifically the curved rind of the sliced melon and the rounded geometry of the surrounding fruit. These well-defined foreground edges provide an ideal geometric reference for tracing multi-segment C^1-continuous cubic Bézier curves and Catmull–Rom splines in the plane of the painting, while offering a rich backdrop for placing 3D objects and evaluating transformation hierarchies.

PART A: Transformations



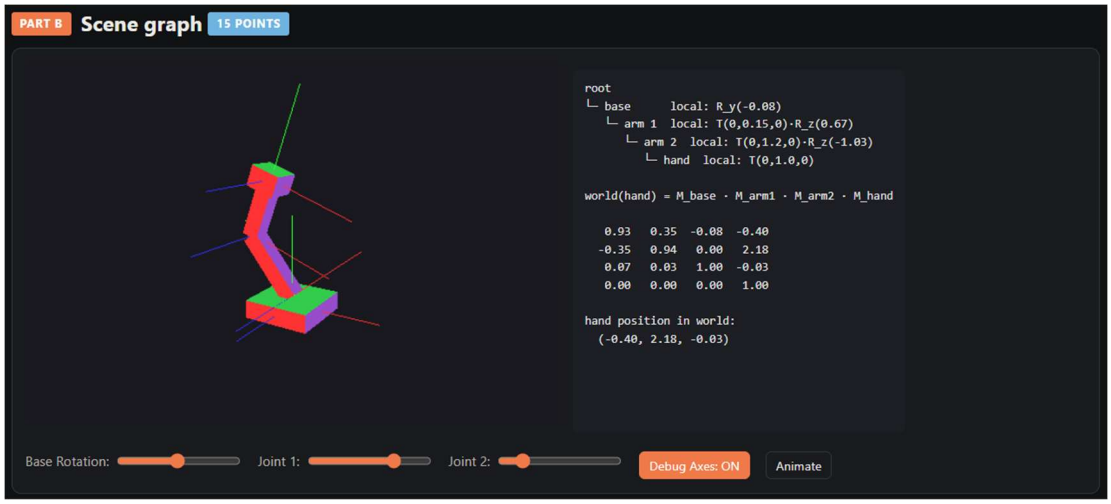
All 4×4 matrices in math/mat4.js use a column-major Float32Array(16) layout matching WebGPU uniform buffers. Because matrix multiplication is non-commutative ($A \cdot B \neq B \cdot A$), applying transformations right-to-left yields distinct results:

- $M = T \cdot R_y \cdot S$ **(Standard Convention):** Scales the object in its local space (S), rotates it in place around its local origin (R_y), and translates it to world coordinates (t_x, t_y, t_z) without distorting the translation offset.
- $M = S \cdot R_y \cdot T$ **(Reversed Convention):** Translates the object first (T), causing subsequent rotation (R_y) to orbit the object around the *world origin* at radius $||\mathbf{t}||$, while scaling (S) scales both the geometry and the rotated translation vector itself:

$$M_{TRS} = \begin{bmatrix} s\cos\theta & 0 & s\sin\theta & t_x \\ 0 & s & 0 & t_y \\ -s\sin\theta & 0 & s\cos\theta & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad M_{SRT} = \begin{bmatrix} s\cos\theta & 0 & s\sin\theta & s(t_x\cos\theta + t_z\sin\theta) \\ 0 & s & 0 & st_y \\ -s\sin\theta & 0 & s\cos\theta & s(-t_x\sin\theta + t_z\cos\theta) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

All subsequent modules in this project use the $M = T \cdot R \cdot S$ convention.

PART B: Scene Graph



SceneNode stores localMatrix, worldMatrix, parent, and children, updating recursively via $M_{\text{world}}(\text{node}) = M_{\text{world}}(\text{parent}) \cdot M_{\text{local}}(\text{node})$.

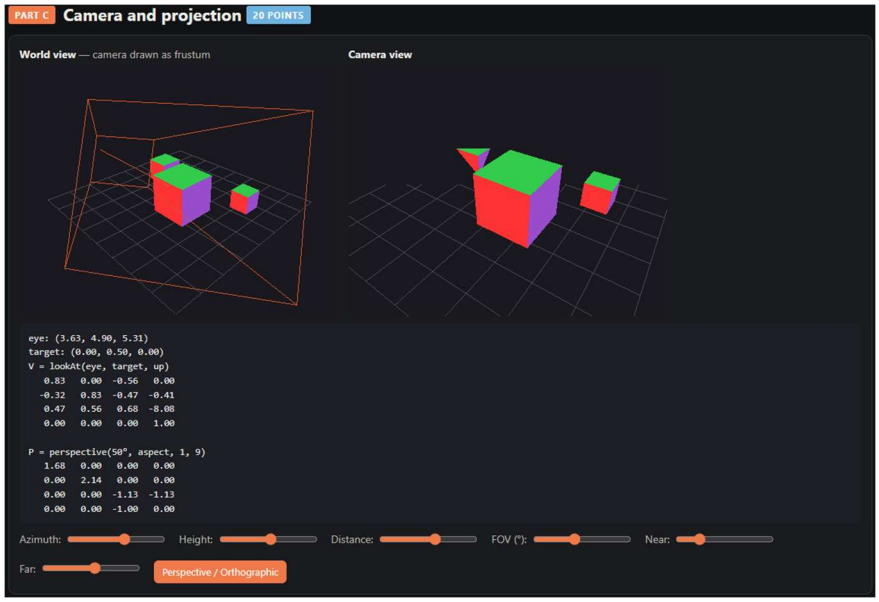
Hierarchy Tree (5 Nodes, 4 Levels): root (I) → base: T(0, 0.15, 0)·Ry(a1) → arm 1: T(0, 0.15, 0)·Rz(a2) → arm 2: T(0, 1.2, 0)·Rz(a3) → hand: T(0, 1.0, 0).

Hand Verification (hand leaf at $a_1 = \pi, a_2 = 0.71, a_3 = -0.77$):

1. **base:** $\cos\pi = -1, \sin\pi = 0 \Longrightarrow M_{\text{base}} = \text{diag}(-1,1,-1,1)$ with translation $(0,0.15,0)^T$.
2. **arm 1:** Translating $+0.15$ along Y gives $(0,0.30,0)^T$. Rotating by $a_2 = 0.71$ ($\cos 0.71 \approx 0.758, \sin 0.71 \approx 0.652$) in the XY-plane negated by M_{base}'s X-flip gives Y-axis column $(0.652,0.758,0)^T$.
3. **arm 2:** Translating 1.2 along arm 1's Y-axis adds $1.2(0.652,0.758,0)^T = (0.782,0.910,0)^T$, placing the joint at $(0.782,1.210,0)^T$. The net Z-angle is $a_2 + a_3 = -0.06$ rad ($\cos(-0.06) \approx 0.998, \sin(-0.06) \approx -0.060$).
4. **hand:** Translating 1.0 along arm 2's Y-axis $(-0.060,0.998,0)^T$ yields $(0.782 - 0.060, 1.210 + 0.998, 0) = (0.722,2.208,0)$, matching the live UI readout (0.72, 2.21, 0.00):

$$M_{\text{world}}(\text{hand}) = M_{\text{base}} \cdot M_{\text{arm1}} \cdot M_{\text{arm2}} \cdot M_{\text{hand}} = \begin{bmatrix} -1.00 & -0.06 & 0.00 & 0.72 \\ -0.06 & 1.00 & 0.00 & 2.21 \\ 0.00 & 0.00 & -1.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 1.00 \end{bmatrix}$$

PART C: Camera and Projection



View Matrix Derivation (lookAt): Given $\mathrm{e} = \text{eye}$, $\mathrm{t} = \text{target}$, and $\mathrm{u}_{\text{up}} = \text{up}$, the camera looks down $-\mathrm{w}$. Using two cross products (with a fallback up vector if $|\mathrm{w} \cdot \mathrm{u}_{\text{up}}| > 0.999$):

$$\mathrm{w} = -\frac{\mathrm{t}-\mathrm{e}}{\|\mathrm{t}-\mathrm{e}\|}, \quad \mathrm{r} = \frac{\mathrm{u}_{\text{up}} \times \mathrm{w}}{\|\mathrm{u}_{\text{up}} \times \mathrm{w}\|}, \quad \mathrm{v} = \mathrm{w} \times \mathrm{r}, \quad V = R_{\text{cam}}^{T} \cdot T(-\mathrm{e})$$

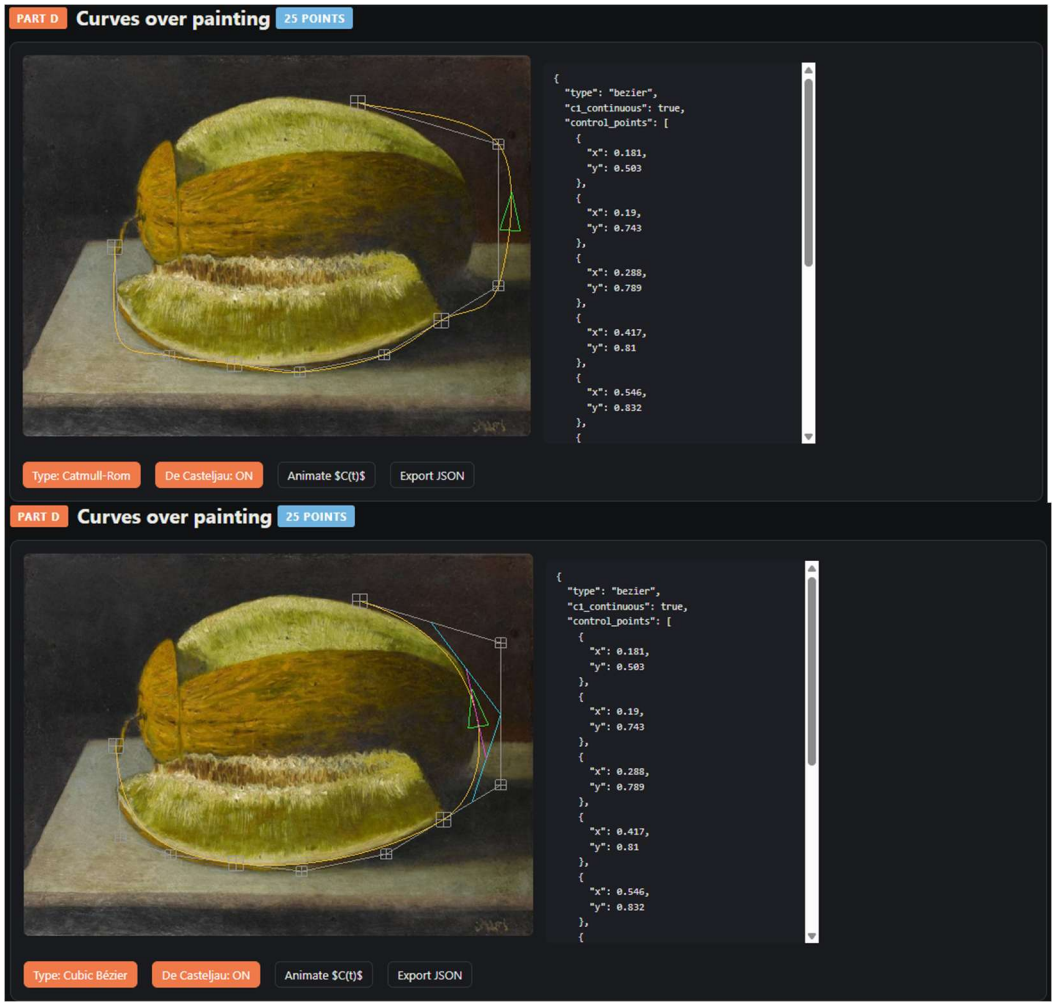
$$= \begin{bmatrix} r_x & r_y & r_z & -(\mathrm{r}\cdot\mathrm{e}) \\ v_x & v_y & v_z & -(\mathrm{v}\cdot\mathrm{e}) \\ w_x & w_y & w_z & -(\mathrm{w}\cdot\mathrm{e}) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

WebGPU Projections ($z_{\text{ndc}} \in [\mathbf{0}, \mathbf{1}]$): With focal length $c = \cot(\text{fovY}/2)$, aspect a, and clipping planes $z_e \in [-n, -f]$, mapping $-n \mapsto 0$ and $-f \mapsto 1$ yields:

$$P_{\text{persp}} = \begin{bmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{f}{n-f} & \frac{nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}, \quad P_{\text{ortho}} = \begin{bmatrix} \frac{1}{r} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & \frac{1}{n-f} & \frac{n}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Depth Precision Discussion: Because perspective depth $z_{\text{ndc}}(z_e) = \frac{f}{n-f} - \frac{nf}{(n-f)z_e}$ scales with $1/z_e$, setting near too close to 0 concentrates almost all depth24plus precision near the camera and causes Z-fighting on distant geometry. Using $\text{near} = 1.0$ and $\text{far} = 9.0$ tightly bounds the three boxes ($f/n = 9$), preserving uniform depth resolution.

PART D: Curves over Painting



Cubic Bézier Basis & Endpoint Proof:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3, \quad t \in [0,1]$$

At $t = 0$, $(1-0)^3 = 1$ while all terms with factor t vanish, proving $B(0) = P_0$. At $t = 1$, $1^3 = 1$ while all terms with factor $(1-t)$ vanish, proving $B(1) = P_3$.

C^1 Continuity at Joints: Using the derivative $B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2)$, two segments $(P_0 . P_3)$ and $(P_3 . P_6)$ meeting at interior knot P_3 have matching tangents ${B_1}'(1) = {B_2}'(0)$ iff:

$$3(P_3 - P_2) = 3(P_4 - P_3) \Longrightarrow \mathbf{P_4 = 2P_3 - P_2} \quad \text{(and } \mathbf{P_7 = 2P_6 - P_5} \text{ at knot } P_6\text{)}$$

PART E: Performance, Debugging & Appendices

- Measured Performance (i9 14900k, RTX 4060ti, 32gb DDR5 RAM): Frame time: **~6.95 ms (144 FPS)**; Total submitted vertices: **~1,170 vertices/frame** (Part A: 216, Part B: 144–168, Part C: 314, Part D: ~ 494).
- What Went Wrong & Fixes: (1) Extracting UI functions into ui/ui.js initially caused duplicate identifier SyntaxErrors until legacy local declarations in app.js were removed. (2) Calling setupPartA before await adapter.requestDevice() threw a ReferenceError halting Parts C–D; moving all initialization calls below pipeline creation resolved it.

Appendix A — Exported Normalized JSON:

{"type": "bezier", "c1_continuous": true, "control_points": [

{"x": 0.181, "y": 0.503}, {"x": 0.19, "y": 0.743}, {"x": 0.288, "y": 0.789}, {"x": 0.417, "y": 0.81},

{"x": 0.546, "y": 0.832}, {"x": 0.712, "y": 0.786}, {"x": 0.824, "y": 0.696}, {"x": 0.937,"y": 0.605},

{"x": 0.937, "y": 0.234}, {"x": 0.66, "y": 0.124}

]

}

Appendix B — AI Declaration: *Google Gemini* was used to cross-verify rubric compliance (adding multi-object draw calls in Part A, RGB debug axes in Part B, a 3rd Bézier segment in Part D, and the Part E Perf counter), debug ES module/WebGPU initialization errors, and format report derivations